Product Requirements Document (PRD)

Street Food Business Operations & Onboarding App

Technical Execution Strategy by Webiq • Confidential • July 2026

1. Executive Summary

This document outlines the functional and technical requirements for a unified mobile and web application designed to manage staff operations for a street food business. The application serves two primary roles: the **Admin** (Business Owner) and the **Chef** (Staff). The core objectives are to digitize the KYC/onboarding process (Aadhaar, PAN) and facilitate daily attendance tracking through a secure, centrally managed platform.

2. Architecture & Technology Stack

> **Unified Codebase Strategy:** The application leverages a single cross-platform framework to serve both mobile (Chef) and web (Admin) interfaces, paired with a robust backend architecture.

- **Frontend / Client:** Flutter (Mobile compilation for Chef UI, Web compilation for Admin Dashboard).
- **Backend API:** NestJS (Node.js/TypeScript framework for business logic, 2FA routing, and secure API gateways).
- **Database & Auth:** Supabase (PostgreSQL for relational data, Supabase Auth for session management, Supabase Storage for secure document holding).
- **2FA / SMS Gateway:** Indian DLT-compliant SMS provider (e.g., MSG91, 2Factor) integrated via NestJS.

3. User Roles & Access Management

Role	Primary Interface	Key Permissions
Admin	Flutter Web / Desktop	Create Chef accounts, toggle 2FA requirements, view uploaded KYC documents, monitor attendance logs.
Chef	Flutter Mobile App	Log in (via Admin-provided credentials & OTP), complete onboarding, upload documents, mark daily attendance.

4. Functional Requirements

4.1 Admin Workflows

- **Chef Account Generation:** The Admin must be able to create an initial account profile for a new Chef by entering basic details (Name, Phone Number). The system will generate a secure initial login ID and password.
- **2FA Configuration:** The Admin dashboard must include a toggle switch for each Chef profile to enable or disable Two-Factor Authentication (OTP via SMS) for their subsequent logins.
- **Document Verification:** The Admin must have a dedicated view to securely open and review KYC documents (Aadhaar, PAN, JPEGs, PDFs) uploaded by the Chef.
- **Attendance Dashboard:** A real-time log displaying attendance records (check-in timestamps) for all active Chefs.

4.2 Chef Workflows

- **Authentication:** Chefs log in using the credentials provided by the Admin. If the Admin has toggled 2FA to 'ON', the NestJS backend triggers an OTP to the Chef's registered mobile number before granting system access.
- **Digital Onboarding (KYC):** Upon first login, if onboarding is incomplete, the Chef is presented with a structured upload flow.
  - Capture or upload Aadhaar Card (Front/Back).
  - Capture or upload PAN Card.
  - Support for both image formats (JPEG/PNG) and documents (PDF).
- **Attendance Marking:** A simplified, highly accessible UI button allowing the Chef to mark their daily attendance once onboarding is successfully completed and approved.

5. Data & Storage Structure

The Supabase integration will utilize the following high-level structure:

- **Database Tables:** `users` (managed via Supabase Auth), `chefs_profiles` (linking auth ID to business logic, holding the `is_2fa_enabled` flag), `attendance_logs` (timestamped records), and `document_metadata`.
- **Storage Buckets:** A private, secure Supabase Storage bucket named `kyc_documents`. Row Level Security (RLS) policies will ensure that a Chef can only write to their own folder, while only the Admin role can read all folders.

6. Security & Compliance

- **KYC Data Protection:** All uploaded files (Aadhaar, PAN) must be transmitted via HTTPS and stored in private buckets. Public URLs must never be generated; access is strictly via signed URLs requested through the NestJS backend.
- **DLT Compliance:** All OTP messages sent to Indian phone numbers must use TRAI-approved DLT templates to ensure high deliverability and avoid carrier blocking.
- **Role-Based Access Control (RBAC):** Strict enforcement on the NestJS backend to ensure Chefs cannot access Admin endpoints, regardless of UI manipulation.